

Charles Dang

CMSC 471, Section 3

September 15, 2026

PA01 Written Section

Generative AI Use:

1. Claude Opus 5 in Visual Studio Code
 - a. Gemini 3.5 Flash-lite for Question 4
2. Code and Answer review and validation
3. Verified results through testing and the provided code snippets (p8.py and wj.py) and reasoning through lecture notes/readings.

Question 1: Heuristic design and admissibility

Heuristic 1: steps

1.

```
wrong = 0
for i in range(len(state)):
    if state[i] != goal[i]: wrong += 1
return wrong
```
2. The lower bound of this heuristic is the number of “wrong” positions. This unavoidable cost is because one action can fix at most 1 wrong position (since it can rewrite only one letter and steps make it so actions cost 1), so you need at least “wrong” number of actions/cost.
3. Every wrong position must be written at least once because otherwise that letter would never change and the final word wouldn't be the goal. So the path takes at least “wrong” actions and therefore costs at least “wrong” number. That holds for every path, including the cheapest one, so $h(n)$ cannot overshoot the true optimal remaining cost.

Heuristic 2: scrabble

1. `total = 0`
for `i` in `range(len(state))`:
 if `state[i] != goal[i]`: `total += scrabble[goal[i]]`
return `total`
2. An action costs the value of the letter it changed to, so just to put the correct letters in place, ignoring any extra moves needed to stay on legal words, you need at least the sum of `scrabble[goal[i]]` over the wrong positions. That is the unavoidable cost.
3. For each wrong position `i`, the final letter there is `goal[i]`, so some action on the path wrote `goal[i]` at position `i`, otherwise it never reaches the goal. That action costs exactly `scrabble[goal[i]]`, and no two wrong positions can share the same action. So the path's total cost is at least the sum of `scrabble[goal[i]]` over the wrong positions, which is $h(n)$. That's true for every path, so also true for the cheapest one, so $h(n)$ never overshoots the true optimal remaining cost.

Heuristic 3: frequency

1. `wrong = 0`
for `i` in `range(len(state))`:
 if `state[i] != goal[i]`: `wrong += 1`
if `wrong == 0`: return 0
return `(wrong - 1) * (1 + min_rarity) + 1 + dictionary[goal]`
2. An action costs `1 + dictionary[new_word]`, so an action costs at least `1 + min_rarity`. It needs a minimum of at least `wrong` number of actions, since one action fixes at most one wrong position. The last one has to land on the goal word, so it costs exactly `1 + dictionary[goal]`, and the other `wrong - 1` each cost at least `1 + min_rarity`. That sum is the unavoidable cost.
3. Any goal path needs at least “wrong” number of actions. The last action produces the goal word, costing exactly `1 + dictionary[goal]`, and each of the other `wrong - 1` or more actions produces some dictionary word, so it costs at least `1 + min_rarity`. So the path costs at least `(wrong - 1) * (1 + min_rarity) + 1 + dictionary[goal]`, which is $h(n)$. True for every path, so also true for the cheapest one, so $h(n)$ never overshoots true optimal.

Question 2: Does the heuristic actually help?

Word Pair	Cost Measure	UCS Cost	UCS Nodes Expanded	A* Cost	A* Nodes Expanded	Reduction
WHY -> ASK	steps	5	773	5	16	97.9%
WHY -> ASK	scrabble	9	765	9	13	98.3%
WHY -> ASK	frequency	38.8554	808	38.8554	634	21.5%
BEAR -> DUCK	steps	4	691	4	6	99.1%
BEAR -> DUCK	scrabble	11	2981	11	6	99.8%
BEAR -> DUCK	frequency	38.2402	1084	38.2402	301	72.2%

1. UCS and A* did not have any discrepancies since all three heuristics do not overshoot the remaining cost. In all six runs, UCS and A* returned the same optimal cost.
2. The "scrabble" cost measure produced the largest reduction in both word pairs. For the "why ask" pair, this is because the scrabble heuristic gives A* more information than steps, so A* expanded fewer nodes (13 vs 16). For the "bear duck" pair, A* expanded the same 6 nodes under both measures; scrabble's larger reduction came from UCS expanding far more nodes (2981 vs 691), since the higher path cost forces UCS to search a much larger region before reaching the goal.
3. For both word pairs, the frequency heuristic was the least informative. For why -> ask, it reduced expansions by only 21.5%, compared with 97.9% for steps and 98.3% for Scrabble. For bear -> duck, it reduced expansions by 72.2%, compared with 99.1% for steps and 99.8% for Scrabble.
4. Steps only counts mismatching positions, while scrabble also accounts for the unavoidable cost of inserting the required goal letters. Frequency uses the global minimum rarity as the lower bound for most remaining actions. Actual words can have much larger rarity values than the global minimum, making the lower bound loose and causing A* to expand more nodes. This is especially visible for why -> ask, where frequency A* expands 634 nodes compared with only 16 for Steps and 13 for Scrabble.

Question 3: Reversibility and direction-dependent costs

Heuristic 1: Steps

1. Yes.
2. Actions cost 1 regardless of the letter, so as long as $W1$ and $W2$ are legal words and changes always cost 1, the cost is symmetric since the optimal forward path is the same length as the optimal reverse path.

Heuristic 2: Scrabble

1. No.
2. Actions cost the letter value of the letter being slotted into the new word. This means that action costs that lead from $W1 \rightarrow W2$ are not the same cost going the other way.
3. $CAT \rightarrow CAB$ costs 3, while in reverse, $CAB \rightarrow CAT$ costs 1.

Heuristic 3: Frequency

1. No.
2. Action cost is $1 + \text{rarity of new word}$. Going $W1 \rightarrow W2$, you pay for $W2$ but not $W1$, and in reverse, going from $W2 \rightarrow W1$, you pay for $W1$ but not $W2$. Every path reverses to a legal path, so the reversed cost is just the forward cost minus $\text{rarity}(W2)$ plus $\text{rarity}(W1)$. The two optimal costs are equal only when the two words are equally rare, which may not always be the case.
3. $CAT \rightarrow COT$ costs ~ 11 , while in reverse, $COT \rightarrow CAT$ costs ~ 8

Reversibility means the reverse action is always legal, so if $W1$ can become $W2$, $W2$ can always become $W1$. This doesn't guarantee the reverse cost is the same as the forward cost. Under steps the cost happens to not care about direction, so reversibility has cost symmetry. Under scrabble and frequency, the cost depends on what the future word is, so the same reversed path carries a different price, which causes the optimal costs to differ.

Question 4: LLM heuristic audit

1. Model: Gemini 3.5 Flash-Lite

Exact Prompt: In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary. The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal. By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

Proposed Heuristic: Estimates the remaining cost from the current word n to the goal by

combining the number of letter changes still needed with rarity costs.

2. Code:

```
h_0 = sum(1 for c1, c2 in zip(state, goal) if c1 != c2)
if h_0 == 0:
    return 0.0
global_min_rarity = min(dictionary.values()) if dictionary else 0.0
neighbors = get_valid_neighbors(state, dictionary)
if neighbors:
    min_next_rarity = min(dictionary[neighbor] for neighbor in neighbors)
else:
    min_next_rarity = global_min_rarity
first_step_cost = 1.0 + min_next_rarity
remaining_steps_cost = (h_0 - 1) * (1.0 + global_min_rarity)

return first_step_cost + remaining_steps_cost
```

3. Proof

Let wrong be # of positions where word n differs from the goal. Each action changes one letter, so it fixes at most one mismatch, so any path from n to the goal needs wrong # of actions

The first action must enter one of n's neighbors, so it costs at least 1 plus the smallest rarity among those neighbors. The other (wrong - 1) actions each enter some dictionary word, so each costs at least 1 plus the global minimum rarity.

Adding those lower bounds gives exactly $H_1(n)$. Since it's a lower bound on every path, it's a lower bound on the best one, so h_1 never overestimates.

(If n has no neighbors, the goal is unreachable and the true cost is infinite, so any finite value is fine.)

$H_1 \geq h_0$, because h_1 equals h_0 plus a sum of rarities, and rarities are never negative. Every rarity in this dictionary is strictly positive, so $h_1 > h_0$ at every non-goal state.

4. It is valid.

5. Solution costs are identical in all tested cases. Gemini's proposed heuristic found the same optimal costs. Expansions reduced by 15% on why -> ask (755 -> 641) and 33% on bear -> duck (720 -> 480). The evidence supports using h_1 over h_0 on these two test cases: both returned the same optimal solution costs, while h_1 reduced expansions from 755 to 641 for why to ask and from 720 to 480 for bear to duck. These experiments support its usefulness, while the admissibility proof provides the general correctness argument.